Anything below this layer could be treated as a black box for learning efficiency reasons.

The List of ***Black boxes*** that the ***BIOS*** is based on:

1- PHYSICS ⇒ *electricity, magnetism*
2- PHYSICAL HARDWARE ⇒ *transistors, capacitors, magnetic platters*
3- HARDWARE CONTROLLERS ⇒ *disk controller, GPU controller, USB controller*
4- DEVICE FIRMWARE ⇒ *GPU firmware, disk firmware, NIC firmware*

---

1. What The BIOS Is

1.1 — The Role of BIOS: What Problem It Solves

Let's start with the fundamental problem.

You press the power button. The CPU wakes up. RAM is empty. The CPU has no instructions to execute. The hard drive has your OS on it but the CPU has no idea how to talk to a hard drive. The GPU is sitting there but the CPU has no idea how to initialize it. The keyboard exists but the CPU has no idea how to read from it.

Everything is connected but nothing knows how to talk to anything else.

Someone needs to go first. Someone needs to wake everything up, initialize it, test it, and set up a basic environment before any OS can run.

That someone is the BIOS.

The BIOS is the first software that runs on your computer. Its job is to initialize all hardware and provide a basic set of services that software above it can use.

Without the BIOS your computer would be a collection of expensive components that have no idea how to work together.

1.2 — Where The BIOS Physically Lives

The BIOS is not on your hard drive. It is not in RAM. It lives on a dedicated chip on your motherboard called a **ROM chip** — Read Only Memory.

This chip is soldered directly onto the motherboard. It holds its data permanently without power — it is non-volatile. You can unplug your computer for 10 years and the BIOS will still be there when you plug it back in.

The BIOS chip is typically 4MB to 32MB in size. Small by modern standards but more than enough to hold the firmware code and data it needs.

Modern systems use a type called **Flash ROM** — a special ROM that can be rewritten under controlled conditions. This is how BIOS updates work. You download a new BIOS version, run an updater, and it carefully rewrites the chip. During normal operation it behaves as read-only.

1.3 — How The CPU Finds The BIOS: The Reset Vector

Here is the critical question — how does the CPU know where the BIOS is?

The answer is hardwired into the CPU silicon at the factory. When power is applied or the CPU resets, it always jumps to one specific address:

0xFFFFFFF0

This is called the **reset vector**. It never changes. Every x86 CPU ever made jumps here on power on.

Now here is the clever part. The motherboard is wired so that address 0xFFFFFFF0 maps to the BIOS ROM chip — not to RAM. When the CPU jumps to 0xFFFFFFF0 it is actually reading from the BIOS chip.

```
CPU powers on
    ↓
CPU jumps to 0xFFFFFFF0 (hardwired)
    ↓
0xFFFFFFF0 is mapped to BIOS ROM chip
    ↓
CPU starts executing BIOS code
```

The instruction sitting at 0xFFFFFFF0 is almost always a JMP — it immediately redirects the CPU to the main body of BIOS code elsewhere in the ROM.

This is elegant engineering. The CPU needs no configuration, no setup, no prior knowledge of where anything is. It just jumps to a fixed address and the hardware makes sure the BIOS is there waiting.

1.4 — BIOS vs UEFI: The Two Worlds

There are two completely different firmware standards you need to know about.

BIOS — The Old World

BIOS stands for **Basic Input Output System**. It dates back to 1975. It was designed for the original IBM PC and has barely changed since.

Age: 1975 — ancient
Written in: 16-bit assembly
Boot method: loads 512 bytes from MBR
Services: provided via interrupts in Real Mode only
Disk limit: 2.2TB maximum
Partitions: 4 maximum (MBR partition table)
Security: zero — anything can run
Interface: text based setup screen

Despite its age, BIOS is still emulated by virtually every PC today for backwards compatibility.

UEFI — The New World

UEFI stands for **Unified Extensible Firmware Interface**. It was standardized in 2005 as a complete replacement for BIOS.

Age: 2005 — modern
Written in: C
Boot method: finds .EFI file on FAT32 partition
Services: full API — Boot Services and Runtime Services
Disk limit: effectively unlimited (GPT partition table)
Partitions: 128 maximum
Security: Secure Boot — only signed code runs
Interface: graphical setup screen with mouse support

UEFI is a completely different beast from BIOS. Your bootloader is literally a PE executable — the same binary format as Windows programs. UEFI loads it from a special partition called the EFI System Partition.

Which One Matters For You Right Now

Both. But we start with BIOS because:

BIOS is simpler to understand
BIOS teaches fundamentals better
BIOS has been documented for 40 years
Most OS dev tutorials start with BIOS
UEFI builds on concepts BIOS teaches

1.5 — Why BIOS Still Matters on Modern Computers

You might wonder — if UEFI replaced BIOS in 2005, why are we learning BIOS?

Several reasons:

Legacy compatibility Every modern UEFI system includes a **CSM — Compatibility Support Module**. This is a full BIOS emulator built into the UEFI firmware. It lets modern computers boot old BIOS bootloaders. Your QEMU setup uses SeaBIOS — an open source BIOS implementation — which is why your bootloader works.

Conceptual foundation BIOS concepts — Real Mode, interrupt services, the IVT, the MBR — are the foundation that everything in OS development builds on. Understanding BIOS makes UEFI, Protected Mode, and everything else make sense.

Simplicity BIOS booting is conceptually simple. 512 bytes, jump to it, done. UEFI booting involves PE executables, FAT32 filesystems, Boot Services APIs — much more complex. Learning BIOS first lets you focus on OS concepts without fighting UEFI complexity.

Still widely used Many embedded systems, virtual machines, and older hardware still use legacy BIOS. Understanding it is genuinely useful.

2. POST — Power On Self Test

2.1 — What POST Is and Why It Exists

When power is applied to your computer, every component starts from a completely unknown state. The RAM has random charge in its capacitors. The GPU has no idea what resolution to use. The disk controller has no idea what drives are connected. Everything is in chaos.

Before any software can run reliably, someone needs to:

1. Test that everything is working correctly
2. Initialize every component into a known state
3. Configure components so they can talk to each other

That someone is POST — Power On Self Test.

POST is the BIOS's initialization sequence. It runs before anything else — before your bootloader, before your OS, before anything. It is the BIOS systematically waking up every piece of hardware in a specific order and making sure everything is ready.

You have seen POST happen without knowing it. When you turn on a computer and see the manufacturer logo before the OS loads — that is POST running. The logo appears the moment the GPU is initialized, but POST continues working in the background until everything is ready.

2.2 — The Exact Sequence of Hardware Initialization

POST follows a strict order. Each step depends on the previous one being complete. You cannot initialize the GPU before the PCI bus. You cannot test RAM before the memory controller. Everything has dependencies.

Here is the complete sequence:

Step 1 → CPU initialization
Step 2 → Chipset initialization
Step 3 → RAM initialization and testing
Step 4 → PCI/PCIe bus scanning

Step 5 → GPU initialization
Step 6 → Storage initialization
Step 7 → Other device initialization
Step 8 → Build system tables
Step 9 → Search for bootable device

Let's go through each one in detail.

2.3 — Step 1: CPU Initialization

The very first thing POST does is verify the CPU itself is functional.

The CPU just jumped from 0xFFFFFFF0 to the main BIOS code. But before doing anything else the BIOS needs to know the CPU is working correctly.

It runs a series of self tests on the CPU:

Tests internal registers — can they hold values correctly?
Tests the ALU — can it add, subtract, compare correctly?
Tests the cache — is L1/L2/L3 cache functioning?
Checks CPU identification — what CPU is this? What features does it have?

The BIOS reads the CPU's CPUID — a special instruction that returns information about the CPU:

Manufacturer → Intel or AMD
Family → which CPU generation
Model → which specific CPU
Features → does it support 64-bit? SSE? AVX?

This tells the BIOS exactly what it is dealing with so it can configure everything correctly for that specific CPU.

The CPU also starts in a very specific state after power on:

Mode: 16-bit Real Mode
IP: 0xFFF0 (points to reset vector)
Flags: mostly cleared
Cache: disabled
Paging: disabled

The BIOS is responsible for further configuring the CPU as needed.

2.4 — Step 2: Chipset Initialization

Before RAM can be used or any device can be talked to, the **chipset** must be initialized.

The chipset is a set of chips on the motherboard that manage all communication between the CPU, RAM, and devices. Think of it as the traffic management system for your entire computer.

Modern systems have two main chipset components:

Northbridge (or MCH — Memory Controller Hub)
→ manages CPU ↔ RAM communication
→ manages CPU ↔ GPU communication
→ on modern CPUs this is built into the CPU itself

Southbridge (or ICH — I/O Controller Hub)
→ manages everything else
→ USB ports, SATA ports, audio, PCIe slots
→ now called PCH (Platform Controller Hub) on Intel

The BIOS programs the chipset registers — setting up timing, bus speeds, memory channels, and routing. Without this initialization nothing else can communicate.

This is done entirely through I/O ports — writing specific values to specific port addresses to configure the chipset hardware.

2.5 — Step 3: RAM Initialization and Testing

This is one of the most critical and time consuming parts of POST.

Phase 1 — Memory Controller Setup

Before RAM can be used the memory controller must be configured. It needs to know:

How many RAM sticks are installed?
What type are they? (DDR4, DDR5)
What speed do they run at?
How many channels? (single, dual, quad)
What are the timing parameters? (CAS latency, RAS to CAS delay)

The BIOS reads this information from a small chip on each RAM stick called **SPD — Serial Presence Detect**. The SPD chip contains the RAM's specifications — the exact timing parameters the memory controller needs to run that RAM correctly.

Phase 2 — RAM Testing

Once the memory controller is configured the BIOS tests the RAM to make sure it actually works.

It writes patterns to RAM and reads them back:

Write 0x55555555 to every address → read it back → matches? ✅
Write 0xAAAAAAAA to every address → read it back → matches? ✅
Write 0x00000000 to every address → read it back → matches? ✅
Write 0xFFFFFFFF to every address → read it back → matches? ✅

If any address returns a different value than what was written — that memory location is faulty.

This is why bad RAM produces beep codes. The BIOS detected that writing to certain addresses and reading back gives wrong results. Since it has no screen yet (GPU not initialized) it communicates through beeps.

Phase 3 — Memory Map Construction

After testing the BIOS builds a map of all available memory — which regions are good RAM, which are reserved, which are memory mapped devices. This map gets stored and eventually passed to your bootloader and kernel.

2.6 — Step 4: PCI/PCIe Bus Scanning

Now the BIOS scans every PCIe lane to discover what devices are connected.

For each device it finds, it reads the device's configuration space — a small block of data built into every PCI device:

Vendor ID → who made this? (0x8086 = Intel, 0x10DE = NVIDIA)
Device ID → which specific device?
Class Code → what type? (display, storage, network)
BAR registers→ what memory and I/O space does it need?

After reading this information the BIOS assigns each device:

A range of memory addresses → so CPU can talk to it via memory mapped I/O
An IRQ number → so it can signal the CPU via interrupts
An I/O port range → for port mapped communication

This assignment process is called **PCI enumeration** and it happens completely automatically. By the time your bootloader runs every device already has its addresses and IRQ numbers assigned.

2.7 — Step 5: GPU Initialization

Now the BIOS initializes the graphics card.

The GPU has its own firmware called the **GPU BIOS** or **VBIOS** — Video BIOS. It is stored in a ROM chip on the graphics card itself.

The BIOS finds the GPU on the PCIe bus, locates its VBIOS in the device's ROM, and executes it. This is called running the **Option ROM**.

The GPU VBIOS:

Initializes the GPU's internal hardware
Sets up the video output — picks a resolution and refresh rate
Initializes the framebuffer — the memory used for the screen image
Sets up the connection to the monitor

After the GPU VBIOS runs the screen turns on. This is exactly the moment your monitor goes from black to showing something. That is why you see the manufacturer logo appear a moment after pressing power — the logo appears the instant the GPU is ready.

The BIOS also sets up a basic video mode — typically **VGA text mode** at 80 columns × 25 rows. This is the mode your bootloader starts in. Each character on screen is stored in video memory at address 0xB8000.

2.8 — Step 6: Storage Initialization

The BIOS now initializes all storage devices — hard drives, SSDs, optical drives, USB drives.

For each storage controller (SATA, NVMe, USB) it:

Initializes the controller hardware
Scans for connected drives
Reads drive identification — model, size, capabilities
Checks for drive firmware updates needed
Adds drive to the list of available boot devices

For hard drives and SSDs the BIOS reads the drive's firmware and establishes communication. The drive firmware handles all the low level details — for HDDs that means managing the read/write heads and platters, for SSDs that means managing flash memory cells and wear leveling.

After this step the BIOS knows exactly what storage is available and can read sectors from any of them.

2.9 — Step 7: Other Device Initialization

The BIOS initializes remaining devices:

Keyboard and Mouse The keyboard controller is initialized. The BIOS sets up the keyboard buffer — a small area in the BIOS Data Area that stores keystrokes until software reads them.

USB Controller USB devices are enumerated. USB keyboards and mice are set up to work as if they were PS/2 devices — this is called USB legacy support and lets old bootloaders use USB keyboards without special drivers.

Audio Basic audio initialization. Most bootloaders never use audio but the hardware needs to be in a known state.

Network If PXE network booting is enabled the network card firmware is initialized for potential network boot.

2.10 — Step 8: Build System Tables

This is one of the most important steps for your OS.

The BIOS builds several data structures in RAM that your bootloader and kernel will use:

The Interrupt Vector Table (IVT)

Built at address 0x0000. 256 entries × 4 bytes = 1KB. Contains the addresses of all BIOS interrupt handlers. Your bootloader uses this to call BIOS functions.

The BIOS Data Area (BDA)

Built at address 0x0400. Contains information about the system:

0x0400 → COM port addresses
0x0408 → LPT port addresses
0x0410 → equipment flags
0x0413 → conventional memory size in KB
0x0417 → keyboard shift flags
0x041A → keyboard buffer head pointer
0x041C → keyboard buffer tail pointer
0x0449 → current video mode
0x044A → number of screen columns
0x0484 → number of screen rows minus 1

ACPI Tables

Advanced Configuration and Power Interface tables — describe the hardware to the OS. Tell the OS about CPU cores, power management, thermal sensors, PCI devices. The kernel uses these extensively.

SMBIOS Tables

System Management BIOS tables — describe the hardware in human readable form. Model number, serial number, memory slots, CPU information. This is where tools like `dmidecode` on Linux get their information.

2.11 — Beep Codes: When POST Fails

If POST finds a problem it needs to communicate the error. But the screen might not be working yet — the GPU might be the faulty component.

So the BIOS uses the system speaker to beep error codes. Different patterns mean different errors:

(AMI BIOS example)
1 beep → RAM refresh failure
2 beeps → RAM parity error
3 beeps → Base memory failure
4 beeps → System timer failure
5 beeps → CPU failure
6 beeps → Keyboard controller failure
8 beeps → GPU memory failure

Every BIOS manufacturer has their own beep code pattern. This is why computer repair people used to carry beep code charts.

If POST succeeds there is usually one short beep — "all good, proceeding to boot."

2.12 — What POST Gives The Bootloader

When POST finishes and your bootloader starts running, here is exactly what has been set up:

CPU — initialized, in 16-bit Real Mode, ready to execute instructions.

RAM — tested, initialized, memory controller configured. You can read and write RAM freely.

IVT — built at 0x0000. All BIOS interrupt handlers are ready. You can call INT 0x10, INT 0x13, INT 0x15 immediately.

GPU — initialized, VGA text mode active, video memory at 0xB8000 ready to use.

Storage — all drives detected and initialized. You can read sectors using INT 0x13.

Keyboard — initialized, keyboard buffer ready.

All PCI devices — assigned memory addresses and IRQ numbers.

DL register — set to the boot drive number (0x80 for first hard disk, 0x00 for first floppy). This is critical — you need this to read more sectors from the disk you booted from.

Section 3 — Memory Layout After POST

From The Absolute Beginning

Start Here — What Is RAM Again?

You already know RAM is a flat array of bytes. Every byte has an address. Think of it like a very long street with millions of houses. Every house has a unique number. Every house holds one byte of data.

House 0 → holds one byte

House 1 → holds one byte

House 2 → holds one byte

...

House 31744 → holds one byte (this is 0x7C00)

...

House 1048575 → holds one byte (this is 0xFFFFF — the last house)

In Real Mode you have exactly 1,048,576 houses — 1MB. That is every address from 0x00000 to 0xFFFFF.

The Problem

When your bootloader starts running, RAM is not empty. The BIOS already moved into some of those houses during POST. It set up important data structures in specific locations.

If your bootloader accidentally writes to a house the BIOS is living in — it corrupts critical data and everything crashes silently.

You need to know exactly which houses are taken and which ones are free.

The Street Map

Here is the complete street from house 0 to house 1,048,575:

Houses 0 to 1,023 → Interrupt Vector Table

Houses 1,024 to 1,279 → BIOS Data Area

Houses 1,280 to 31,743 → FREE — yours to use

Houses 31,744 to 32,255 → Your bootloader Stage 1

Houses 32,256 to 40,959 → FREE — Stage 2 loads here

Houses 40,960 to 49,151 → Video memory

Houses 49,152 to 65,535 → BIOS ROM

Houses 65,536 to 655,359 → FREE — kernel loads here

Houses 655,360 to 1,048,575 → Reserved hardware regions

Now let's walk through each section and understand exactly what it is and why it exists.

House 0 to 1,023 — The Interrupt Vector Table

The very first 1,024 houses of RAM. The BIOS built this during POST.

What It Is

A lookup table. Nothing more. 256 entries sitting one after another. Each entry is 4 bytes — the address of a BIOS function.

Entry 0 at house 0 → 4 bytes → address of handler 0

Entry 1 at house 4 → 4 bytes → address of handler 1

Entry 2 at house 8 → 4 bytes → address of handler 2

...

Entry 16 at house 64 → 4 bytes → address of video handler

Entry 19 at house 76 → 4 bytes → address of disk handler

...

Entry 255 at house 1020 → 4 bytes → address of handler 255

256 entries × 4 bytes = 1,024 bytes = exactly 1KB.

Why It Exists

When your bootloader calls INT 0x10, the CPU needs to know where the BIOS video code lives in memory. It finds that address by looking here — at entry 16 (0x10 in hex). The address stored there is where the CPU jumps.

Without the IVT, software interrupts would have nowhere to look and the CPU would jump to a random address and crash.

The Rule

Never write to houses 0 to 1,023. If you overwrite an entry you corrupt that BIOS function. The next INT call jumps to garbage and crashes.

Houses 1,024 to 1,279 — The BIOS Data Area

The 256 houses immediately after the IVT. The BIOS wrote system facts here during POST.

What It Is

A small collection of facts the BIOS discovered during POST and stored for your bootloader to read:

House 1,024 (0x0400) → COM port address

House 1,043 (0x0413) → how much RAM exists in KB

House 1,097 (0x0449) → current video mode

House 1,098 (0x044A) → screen width in columns (usually 80)

House 1,156 (0x0484) → screen height in rows minus 1 (usually 24)

House 1,050 (0x041A) → keyboard buffer head

House 1,052 (0x041C) → keyboard buffer tail

Why It Exists

Your bootloader sometimes needs quick facts about the system. Instead of calling a BIOS interrupt every time, you can just read directly from the BDA.

For example to find how much RAM exists:

```
mov ax, [0x0413]    ← go to house 1,043, read what is there

                    ax now holds the RAM size in KB
```

One instruction. Instant answer.

The Rule

Never write to houses 1,024 to 1,279. You would corrupt BIOS state and break interrupt handlers.

Houses 1,280 to 31,743 — First Free RAM

The first genuinely free region. Nobody lives here. You can use it however you want.

What Goes Here

The Stack

You already know the stack grows downward. Your bootloader sets SP somewhere near the top of this region — close to house 31,743. As functions are called the stack grows downward into this space:

```
House 31,743  ← SP starts near here

House 31,741  ← push AX: SP moved down, AX stored here

House 31,739  ← push BX: SP moved down, BX stored here

...

House 1,280   ← bottom of free RAM, stack cannot go below here
```

Temporary Data

Any data your bootloader needs while running. Buffers, variables, disk read results. You pick an address and use it:

mov [0x1000], ax ← store AX at house 4,096

mov ax, [0x1000] ← read from house 4,096 back into AX

The Rule

This region is yours. Use it freely. Stack goes here. Temporary data goes here.

Houses 31,744 to 32,255 — Your Stage 1 Bootloader

512 houses. This is where the BIOS loaded your bootloader code.

What It Is

Your boot.asm compiled into machine code bytes sitting in these 512 houses. The CPU is executing instructions from here right now.

House 31,744 (0x7C00) ← your first instruction

House 31,745 (0x7C01) ← second byte of code

House 31,746 (0x7C02) ← third byte

...

House 32,255 (0x7DFF) ← last byte of your 512 byte bootloader

Why Only 512 Houses

Because the BIOS only reads the first 512 bytes of a disk — one sector. It loads those 512 bytes here and jumps to house 31,744. Your code starts running from there.

The Rule

This is your code. The CPU is executing from here. Do not write to this region while running — you would overwrite your own code.

Houses 32,256 to 40,959 — Second Free RAM — Stage 2 Lands Here

About 130KB of free RAM immediately above your bootloader.

What It Is

This is where Stage 2 gets loaded. Remember Stage 1's only job is to read Stage 2 from disk and put it somewhere in RAM. This is that somewhere.

Stage 1 calls INT 0x13 and says:

"Read sectors 1 through 10 from disk

put them at house 32,256 (address 0x7E00)"

The BIOS reads those sectors off the disk and copies the bytes into RAM starting at house 32,256. Stage 2 is now sitting in RAM ready to run.

Stage 1 then jumps to house 32,256 and Stage 2 starts executing.

Why Right After Stage 1

Because it is the cleanest arrangement:

Stage 1 ends at house 32,255

Stage 2 starts at house 32,256

No gap, no wasted space

What Stage 2 Does

Stage 2 is the rest of the bootloader — the part that did not fit in 512 bytes. It does all the real work:

Enables A20 line

Gets memory map from BIOS

Builds the GDT

Switches CPU to Protected Mode

Sets up page tables

Switches CPU to Long Mode

Loads kernel from disk

Jumps to kernel

The Rule

Load Stage 2 here. This is its home.

Houses 40,960 to 49,151 — Video Memory

128KB of addresses that are not real RAM. This region is mapped to the video card.

What It Is

Three sections for different video modes:

Houses 40,960 to 45,055 → VGA graphics framebuffer

Houses 45,056 to 47,103 → Monochrome text mode

Houses 47,104 to 49,151 → Color text mode ← THIS IS WHAT YOU USE

The color text mode starts at house 47,104 — address 0xB8000.

How It Works

Writing to address 0xB8000 does not store data in RAM. The motherboard is wired so that address goes to the video card instead. The video card receives the write and displays it on screen.

Each character on screen takes two consecutive houses:

House 47,104 (0xB8000) → the character (ASCII value)

House 47,105 (0xB8001) → the color

House 47,106 (0xB8002) → next character

House 47,107 (0xB8003) → its color

The screen is 80 columns × 25 rows = 2,000 characters × 2 bytes = 4,000 bytes of screen memory.

To put the letter H in white on black at the top left corner:

mov byte [0xB8000], 0x48 ← H in ASCII

mov byte [0xB8001], 0x0F ← white text, black background

The Rule

Writing here goes to the video card not RAM. Never use for storing your own data.

Houses 49,152 to 65,535 — BIOS ROM and Option ROMs

256KB mapped to ROM chips. Not RAM — read only memory.

What It Is

Houses 49,152 to 51,199 → GPU BIOS (ran during POST)

Houses 51,200 to 57,343 → Other device firmware

Houses 57,344 to 65,535 → Main BIOS code

The BIOS code that ran when you pressed power, initialized everything, built the IVT, found your bootloader — it lives here permanently in ROM.

Why Read Only

These addresses are physically connected to ROM chips on the motherboard — not RAM chips. ROM holds its data permanently without power. It cannot be written to during normal operation.

If you try to write here the write is silently ignored. The data does not change.

The Rule

Never write here. Read only. Writes are ignored.

Houses 65,536 to 655,359 — More Free RAM

About 576KB of free RAM. This is where your kernel gets loaded.

What It Is

A large stretch of free RAM above the ROM region. Stage 2 reads your kernel binary from disk and loads it here.

The kernel can be much larger than Stage 2 — this region gives it plenty of room. As your OS grows more complex and your kernel gets bigger, this space accommodates it.

The Rule

Load your kernel here.

Houses 655,360 to 1,048,575 — Reserved Hardware Regions

The top 384KB of the 1MB space. Various hardware devices are mapped here:

0xA0000 → VGA graphics framebuffer

0xB8000 → VGA text mode (already covered)

0xC0000 → Option ROMs (already covered)

0xF0000 → BIOS entry point area

0xFFFF0 → Reset vector (where CPU jumped on power on)

Nothing here is usable RAM. All hardware mapped or reserved.

The Rule

Never use this region for your own data.

The 1MB Ceiling — Why The Map Ends Here

The entire map ends at house 1,048,575 — 1MB. This is the absolute limit of 16-bit Real Mode.

The address bus in Real Mode is 20 bits wide:

2^{20} = 1,048,576 possible addresses = 1MB

You literally cannot express an address higher than 0xFFFFF in Real Mode. The number does not fit in the registers. Any attempt to access memory above 1MB wraps around back to the bottom.

This is why switching CPU modes is so critical. Protected Mode gives you 4GB. Long Mode gives you effectively unlimited memory. Your Stage 2 bootloader's job includes making that transition.

The Rules In One Place

NEVER TOUCH:

Houses 0 to 1,023 → IVT — corrupting breaks all BIOS calls

Houses 1,024 to 1,279 → BDA — corrupting breaks BIOS state

Houses 40,960+ → video memory, ROM, reserved hardware

YOURS TO USE:

Houses 1,280 to 31,743 → stack and temporary data

Houses 31,744 to 32,255 → your Stage 1 code

Houses 32,256 to 40,959 → Stage 2 loads here

Houses 65,536 to 655,359 → kernel loads here

Section 4 — The Interrupt Vector Table

From The Absolute Beginning

4.1 — What The IVT Is and Why It Exists

The Problem

Your bootloader needs to print to the screen. It needs to read from disk. It needs to get the memory map. But it has no idea how to do any of these things directly — the hardware is too complex and different on every machine.

The BIOS has functions that handle all of this. But your bootloader needs to know WHERE in memory those functions live before it can call them.

This is the problem the IVT solves.

The Solution

During POST the BIOS writes all its function addresses into one central lookup table in RAM. When your bootloader needs a BIOS function it looks up the address in this table and jumps there.

That table is the **Interrupt Vector Table**.

Think of it exactly like a contacts list on your phone:

Contact "Video" → phone number to call for screen operations

Contact "Disk" → phone number to call for disk operations

Contact "Memory" → phone number to call for memory operations

The IVT is that contacts list. Instead of phone numbers it stores memory addresses. Instead of names it uses numbers.

Why At Address 0x0000

The IVT lives at the very first address in RAM — 0x0000. This was a deliberate design decision by Intel and IBM.

The CPU needs to find the IVT instantly when an interrupt fires. If the IVT could be anywhere in memory the CPU would need to search for it — slow and complex.

By hardcoding it at address 0x0000 the CPU always knows exactly where to look. No searching. No configuration. Just go to address 0x0000 and look up the entry.

4.2 — Physical Structure

How Many Entries

The IVT has exactly **256 entries**. This gives you 256 possible interrupt numbers — from 0 to 255. Each interrupt number corresponds to one entry in the table.

How Big Each Entry Is

Each entry is **4 bytes**. Those 4 bytes store the memory address of the handler function for that interrupt.

Why 4 bytes? Because in Real Mode a full memory address needs 4 bytes — 2 bytes for the segment and 2 bytes for the offset. You learned about segment:offset addressing earlier. A complete Real Mode address needs both.

Entry structure (4 bytes):

Bytes 0-1 → offset (2 bytes)

Bytes 2-3 → segment (2 bytes)

Total Size

256 entries × 4 bytes = 1024 bytes = 1KB

The IVT takes exactly 1KB of RAM — from address 0x0000 to address 0x03FF.

How Entries Are Laid Out

Address 0x0000 → entry 0 (4 bytes)

Address 0x0004 → entry 1 (4 bytes)

Address 0x0008 → entry 2 (4 bytes)

Address 0x000C → entry 3 (4 bytes)

...

Address 0x0040 → entry 16 (4 bytes) ← INT 0x10

Address 0x004C → entry 19 (4 bytes) ← INT 0x13

Address 0x0054 → entry 21 (4 bytes) ← INT 0x15

...

Address 0x03FC → entry 255 (4 bytes)

Each entry sits exactly 4 bytes after the previous one. Clean, simple, predictable.

4.3 — How The CPU Calculates Entry Addresses

The Formula

Address of entry N = 0x0000 + (N × 4)

Since the IVT starts at 0x0000 this simplifies to:

Address of entry N = N × 4

Worked Examples

INT 0x10 — Video Service (entry 16)

Address = 16 × 4 = 64 = 0x0040

CPU goes to address 0x0040

Reads 4 bytes

Gets address of BIOS video handler

Jumps there

INT 0x13 — Disk Service (entry 19)

Address = 19 × 4 = 76 = 0x004C

CPU goes to address 0x004C

Reads 4 bytes

Gets address of BIOS disk handler

Jumps there

INT 0x15 — Memory Service (entry 21)

Address = 21 × 4 = 84 = 0x0054

CPU goes to address 0x0054

Reads 4 bytes

Gets address of BIOS memory handler

Jumps there

Why Multiply By 4

Because each entry is 4 bytes wide. To skip N entries you move forward N × 4 bytes. Simple array indexing — exactly the same concept as accessing element N of an array in any programming language.

4.4 — The Complete List of Important Interrupts

The 256 entries are divided into three categories:

CPU Exceptions (Entries 0 to 31)

These are not BIOS functions. They are errors the CPU itself generates when something goes wrong:

INT 0x00 → Divide by zero

happens when you divide a number by zero

INT 0x01 → Debug exception

used by debuggers for single stepping

INT 0x02 → Non-maskable interrupt

critical hardware error that cannot be ignored

INT 0x03 → Breakpoint

used by debuggers to pause execution

INT 0x04 → Overflow

signed arithmetic overflow detected

INT 0x05 → Bound range exceeded

array index out of bounds

INT 0x06 → Invalid opcode

CPU encountered an instruction it does not understand

INT 0x07 → Device not available

tried to use FPU when not available

INT 0x08 → Double fault

two exceptions happened at the same time

INT 0x0D → General protection fault

program tried to do something it is not allowed to

INT 0x0E → Page fault

program accessed a memory page that is not mapped

These fire automatically when the CPU detects an error. You do not call them deliberately.

Hardware Interrupts (Entries 8 to 15 in Real Mode)

These fire automatically when hardware devices need attention:

INT 0x08 → System timer fires 18.2 times per second

INT 0x09 → Keyboard a key was pressed

INT 0x0A → cascade connects to second PIC

INT 0x0B → Serial port 2

INT 0x0C → Serial port 1

INT 0x0D → Parallel port

INT 0x0E → Floppy disk disk operation finished

INT 0x0F → Parallel port 2

These fire on their own when devices need attention. You do not call them deliberately either.

BIOS Software Interrupts (Your Tools)

These are the BIOS functions you call deliberately using the INT instruction:

INT 0x10 → Video services

print characters, set video mode, move cursor

INT 0x11 → Get equipment list

what hardware is installed

INT 0x12 → Get conventional memory size

how much memory below 1MB

INT 0x13 → Disk services

read and write sectors from disk

INT 0x14 → Serial port services

communicate via serial port

INT 0x15 → Miscellaneous services

most importantly — get full memory map (E820)

INT 0x16 → Keyboard services

read keystrokes, check if key available

INT 0x17 → Printer services

talk to parallel port printer

INT 0x19 → Bootstrap loader

reboot the system

INT 0x1A → Real time clock

get current time and date

These are the ones you call deliberately in your bootloader code.

4.5 — How The CPU Uses The IVT

This is the most important part. Let's trace exactly what happens when your bootloader executes INT 0x10.

Step 1 — You Set Up Registers

Before calling the interrupt you put arguments in registers:

mov ah, 0x0E ← which BIOS function you want

mov al, 'H' ← the character to print

Step 2 — You Fire The Interrupt

int 0x10

Step 3 — CPU Saves FLAGS

The CPU immediately pushes the FLAGS register onto the stack:

Stack before: [empty]

Stack after: [FLAGS]

Why? Because the BIOS handler might change flags during its work. Saving them means they get restored when it returns.

Step 4 — CPU Disables Interrupts

The CPU automatically clears the interrupt flag (IF = 0). This prevents other interrupts from firing while this one is being handled.

Step 5 — CPU Saves Return Address

The CPU pushes CS (code segment) and IP onto the stack:

Stack now: [FLAGS][CS][IP]

This is the bookmark — exactly where to come back after the BIOS finishes.

Step 6 — CPU Calculates IVT Entry Address

Entry address = 0x10 × 4 = 0x0040

Step 7 — CPU Reads Handler Address

The CPU goes to address 0x0040 in RAM and reads 4 bytes. Those bytes are the address of the BIOS video handler — wherever the BIOS put it during POST.

Step 8 — CPU Jumps To Handler

The CPU loads the handler address into CS and IP. The BIOS video code starts running.

Step 9 — BIOS Does The Work

The BIOS video handler reads your registers:

AH = 0x0E → print a character

AL = 0x48 → the character is H

It writes H to video memory at 0xB8000. H appears on screen.

Step 10 — BIOS Returns With IRET

The BIOS executes IRET — Interrupt Return. The CPU pops IP, CS, and FLAGS off the stack — restoring exactly where you were:

Stack before IRET: [FLAGS][CS][IP]

Stack after IRET: [empty]

IP and CS restored → back to instruction after INT 0x10

FLAGS restored → exactly as they were before

Your bootloader continues from the instruction right after INT 0x10.

4.6 — How The BIOS Builds The IVT

During POST the BIOS builds the entire IVT before your bootloader ever runs.

The Process

For each interrupt number the BIOS:

1. Writes its handler function somewhere in BIOS ROM or RAM

2. Calculates the IVT entry address (N × 4)

3. Writes the handler address into that IVT entry

By the time POST finishes all 256 entries are filled. Every interrupt has a handler address.

What Happens If An Entry Is Corrupt

If your bootloader accidentally overwrites an IVT entry — say you write random data to address 0x0040 — the next time INT 0x10 fires the CPU reads that garbage address and jumps there. Whatever is at that address gets executed as code. Instant crash with no error message.

This is why the rule is absolute — never write to addresses 0x0000 to 0x03FF.

4.7 — Hardware Interrupts vs Software Interrupts In The IVT

Both types use the same IVT. But they are triggered differently.

Software Interrupts — You Trigger These

Your code deliberately calls them:

int 0x10 ← you chose to call this

You control when they happen. You set up registers first. You know they are coming.

Hardware Interrupts — Devices Trigger These

Devices signal the PIC which signals the CPU. The CPU pauses whatever it is doing and handles it:

You are running code...

Keyboard fires IRQ 1 → CPU gets interrupt 0x09

CPU saves position on stack

CPU looks up entry 0x09 in IVT

CPU jumps to keyboard handler

Handler reads which key was pressed

Handler stores it in keyboard buffer

IRET → CPU returns to exactly where it was

You did not call this. The keyboard triggered it automatically.

Same Mechanism Different Trigger

Software interrupt → your INT instruction triggers it

Hardware interrupt → a device triggers it automatically

Both → CPU uses IVT to find handler address

Both → CPU saves FLAGS, CS, IP on stack

Both → handler runs, IRET returns

The IVT handles both identically. The only difference is what triggered the lookup.

4.8 — Why The IVT Disappears In Protected Mode

This is critical for your bootloader to understand.

The IVT Is a Real Mode Concept

The IVT works because of how Real Mode addressing works. The CPU knows to look at address 0x0000 for interrupt handlers. This is hardcoded into Real Mode behavior.

Protected Mode Uses The IDT Instead

When you switch to Protected Mode the CPU stops using the IVT completely. It switches to a different structure called the **IDT — Interrupt Descriptor Table**.

The IDT is similar in concept but completely different in structure. It can live anywhere in memory — not just 0x0000. You tell the CPU where it is using a special register called IDTR.

What This Means For Your Bootloader

The moment you switch to Protected Mode:

INT 0x10 → no longer calls BIOS video function

INT 0x13 → no longer calls BIOS disk function

INT 0x15 → no longer calls BIOS memory function

All BIOS functions become completely inaccessible. Forever.

This is why Stage 2 must collect everything it needs from the BIOS before switching modes:

Get memory map (INT 0x15) ← must do this in Real Mode

Enable A20 (via ports) ← must do this in Real Mode

Set up video (INT 0x10) ← must do this in Real Mode

Read kernel from disk (INT 0x13) ← must do this in Real Mode

↓

THEN switch to Protected Mode

↓

BIOS is gone forever

Kernel handles everything from here

Section 5 — BIOS Interrupt Services

Your Tools For Talking To Hardware

The Big Picture First

Your bootloader runs in a very primitive environment. No OS. No drivers. No libraries. Just you, the CPU, and the BIOS.

The BIOS is your only helper. It has ready-made functions for everything you need — printing to screen, reading from disk, getting memory information, reading keyboard input.

These functions are called through software interrupts. You put arguments in registers, fire an INT instruction, the BIOS does the work, puts results back in registers, and returns control to you.

There are four interrupt services your bootloader uses constantly:

INT 0x10 → everything related to the screen

INT 0x13 → everything related to the disk

INT 0x15 → everything related to memory

INT 0x16 → everything related to the keyboard

Let's go through each one in complete detail.

5.1 — INT 0x10: Video Services

INT 0x10 handles everything related to the screen. Printing characters, moving the cursor, changing colors, switching video modes.

How You Call It

Every INT 0x10 call follows the same pattern:

1. Put the function code in AH
2. Put any arguments in other registers
3. Fire INT 0x10
4. Read results from registers if needed

Function 0x00 — Set Video Mode

This sets what mode the screen operates in.

Before calling:

AH = 0x00 ← set video mode function

AL = mode ← which mode you want

After calling: screen is now in the requested mode

The most important modes:

AL = 0x03 → 80×25 color text mode ← standard bootloader mode

AL = 0x13 → 320×200 256 color graphics mode

AL = 0x12 → 640×480 16 color graphics mode

Your bootloader starts in mode 0x03 automatically. You rarely need to set this explicitly unless you want graphics.

Example:

```
mov ah, 0x00

mov al, 0x03    ← set 80x25 color text mode

int 0x10
```

Function 0x01 — Set Cursor Shape

Controls how the cursor looks on screen.

Before calling:

AH = 0x01

CH = cursor start scan line (top of cursor)

CL = cursor end scan line (bottom of cursor)

Example — hide the cursor:

```
mov ah, 0x01

mov ch, 0x20    ← bit 5 set = cursor hidden

int 0x10
```

Example — normal underscore cursor:

```
mov ah, 0x01

mov ch, 0x06    ← start line 6

mov cl, 0x07    ← end line 7

int 0x10
```

Function 0x02 — Set Cursor Position

Moves the cursor to a specific position on screen.

Before calling:

AH = 0x02

BH = page number (use 0 for page 0)

DH = row (0 = top, 24 = bottom)

DL = column (0 = left, 79 = right)

Example — move cursor to top left:

```
mov ah, 0x02
mov bh, 0x00    ← page 0
mov dh, 0x00    ← row 0 (top)
mov dl, 0x00    ← column 0 (left)
int 0x10
```

Example — move cursor to row 5, column 10:

```
mov ah, 0x02
mov bh, 0x00
mov dh, 0x05    ← row 5
mov dl, 0x0A    ← column 10
int 0x10
```

Function 0x03 — Get Cursor Position

Reads the current cursor position.

Before calling:

AH = 0x03

BH = page number (use 0)

After calling:

DH = current row

DL = current column

CH = cursor start scan line

CL = cursor end scan line

Example:

```
mov ah, 0x03
mov bh, 0x00
int 0x10
← DH now holds current row
← DL now holds current column
```

Function 0x0E — Print Character (Teletype Mode)

This is the one you already used. Prints one character at the current cursor position and advances the cursor automatically.

Before calling:

AH = 0x0E ← teletype print function

AL = character ← ASCII value of character to print

BH = page number (use 0)

BL = color (only matters in graphics mode)

After calling:

character appears on screen

cursor moves to next position

Example — print the letter H:

```
mov ah, 0x0E
mov al, 'H'     ← ASCII 0x48
mov bh, 0x00
int 0x10
```

Special characters it handles automatically:

0x08 → backspace (moves cursor back one)

0x0A → line feed (moves cursor down one row)

0x0D → carriage return (moves cursor to start of line)

0x07 → bell (makes a beep sound)

Function 0x13 — Print String

Prints an entire string at once. More efficient than printing one character at a time.

Before calling:

AH = 0x13

AL = write mode (0-3, controls cursor movement)

BH = page number (use 0)

BL = color attribute

CX = length of string (number of characters)

DH = row to start printing

DL = column to start printing

ES:BP = address of string in memory

After calling:

entire string printed on screen

The write modes:

AL = 0 → attribute in BL, cursor not moved after print

AL = 1 → attribute in BL, cursor moved after print

AL = 2 → string contains character+attribute pairs, cursor not moved

AL = 3 → string contains character+attribute pairs, cursor moved

Color Attributes

Every character on screen has a color attribute — one byte controlling foreground and background color:

```
Bits 7    6 5 4    3 2 1 0

   |    └─┬─┘    └──┬──┘
   |      |         |
   |      |         └── foreground color (text color)
   |      └──────────────── background color
   └──────────────────────────── blink (1=blinking text)
```

Color values:

0x0 → black

0x1 → blue

0x2 → green

0x3 → cyan

0x4 → red

0x5 → magenta

0x6 → brown

0x7 → light gray

0x8 → dark gray

0x9 → light blue

0xA → light green

0xB → light cyan

0xC → light red

0xD → light magenta

0xE → yellow

0xF → white

Common color attributes:

0x07 → light gray text on black (default)

0x0F → white text on black

0x1F → white text on blue

0x4F → white text on red

0x2F → white text on green

0x0A → light green text on black

0x0E → yellow text on black

INT 0x10 Summary

AH = 0x00 → set video mode

AH = 0x01 → set cursor shape

AH = 0x02 → set cursor position

AH = 0x03 → get cursor position

AH = 0x0E → print one character ← you already used this

AH = 0x13 → print entire string

5.2 — INT 0x13: Disk Services

INT 0x13 handles everything related to reading and writing disk sectors. This is how your Stage 1 bootloader reads Stage 2 from disk. This is how Stage 2 reads your kernel from disk.

Two Addressing Methods

There are two ways to tell the BIOS which sector you want:

CHS — Cylinder Head Sector (old way) Describes physical disk geometry. Cylinder number, head number, sector number. Complex and tied to physical hardware.

LBA — Logical Block Addressing (modern way) Treats the disk as a flat array of sectors numbered from 0. Simple and clean.

Your bootloader uses LBA. Much simpler.

Function 0x00 — Reset Disk System

Resets the disk controller. Call this if a disk operation fails before retrying.

Before calling:

AH = 0x00

DL = drive number (0x80 = first hard disk)

After calling:

CF = 0 if success

CF = 1 if error

Example:

mov ah, 0x00

mov dl, 0x80 ← first hard disk

int 0x13

Function 0x02 — Read Sectors (CHS)

Reads sectors using the old CHS addressing. You will not use this much but good to know it exists.

Before calling:

AH = 0x02

AL = number of sectors to read

CH = cylinder number (low 8 bits)

CL = sector number (bits 0-5) + cylinder high bits (bits 6-7)

DH = head number

DL = drive number (0x80 = first hard disk)

ES:BX = address to load data into

After calling:

CF = 0 if success

CF = 1 if error

AH = error code if failed

AL = number of sectors actually read

Function 0x42 — Read Sectors (LBA Extended)

This is the one you will actually use. Reads sectors using LBA addressing — simple sector numbers from 0.

Before calling:

AH = 0x42

DL = drive number (0x80 = first hard disk)

DS:SI = address of Disk Address Packet in memory

After calling:

CF = 0 if success

CF = 1 if error

AH = error code if failed

The Disk Address Packet

Function 0x42 uses a special structure called a **Disk Address Packet (DAP)**. You build this structure in memory before calling the interrupt:

Offset	Size	Value
0	1	size of DAP (always 0x10 — 16 bytes)
1	1	always 0x00
2	2	number of sectors to read
4	4	address in RAM to load into (segment:offset)

8 8 LBA sector number to start reading from

Example — reading 5 sectors starting from sector 1 into address 0x7E00:

```
dap:
    db 0x10          ← DAP size = 16 bytes
    db 0x00          ← always zero
    dw 0x0005        ← read 5 sectors
    dw 0x7E00        ← offset to load into
    dw 0x0000        ← segment (0x0000 means address = offset)
    dq 0x0000000000000001  ← LBA sector number 1
```

Then call it:

```
mov ah, 0x42
mov dl, 0x80        ← first hard disk
mov si, dap         ← DS:SI points to the DAP structure
int 0x13
```

Checking If It Worked

After any INT 0x13 call you must check the carry flag:

```
jc disk_error       ← CF=1 means error occurred
                    CF=0 means success
```

If CF is set, AH contains an error code:

0x01 → invalid command

0x02 → address mark not found

0x04 → sector not found

0x20 → controller failure

0x40 → seek failed

0x80 → disk timeout (drive not ready)

INT 0x13 Summary

AH = 0x00 → reset disk system

AH = 0x02 → read sectors (CHS — old way)

AH = 0x42 → read sectors (LBA — modern way) ← use this

Always check carry flag after disk operations. Always have error handling.

5.3 — INT 0x15: Memory Services

INT 0x15 is your window into understanding what memory exists on the system. The most important function is E820 — getting the complete memory map.

Why You Need The Memory Map

Your OS needs to manage RAM. But it cannot just assume all RAM is free. Some regions are reserved by hardware. Some are used by BIOS. Some are ACPI data that must not be overwritten.

The E820 memory map tells you exactly:

Which regions of RAM are usable free memory

Which regions are reserved by hardware

Which regions contain ACPI data

Which regions are unusable

Without this map your OS might try to use memory that belongs to hardware — corrupting everything.

Function 0xE820 — Get Memory Map

This function returns the memory map one entry at a time. You call it in a loop until it tells you it is done.

Before first call:

EAX = 0x0000E820 ← function code

EBX = 0x00000000 ← start with 0, continuation value

ECX = 24 ← size of buffer (24 bytes per entry)

EDX = 0x534D4150 ← magic number 'SMAP' in ASCII

ES:DI = address of buffer to store entry

After each call:

EAX = 0x534D4150 ← BIOS confirms with same magic number

EBX = continuation value (0 means this was the last entry)

CX = bytes stored in buffer

CF = 0 if success, 1 if error

The Memory Map Entry Structure

Each entry is 24 bytes describing one region of memory:

Offset Size Description

0	8	base address (where this region starts)
8	8	length (how many bytes in this region)
16	4	type (what kind of memory this is)
20	4	ACPI attributes (extended attributes)

Memory Types

Type 1 → usable RAM ← free to use

Type 2 → reserved ← do not touch

Type 3 → ACPI reclaimable ← ACPI data, can use after ACPI init

Type 4 → ACPI NVS ← ACPI non-volatile, never touch

Type 5 → bad memory ← hardware reported as bad

How To Call It In A Loop

```
get_memory_map:
    mov eax, 0x0000E820     ← function code
    mov ebx, 0              ← start from beginning
    mov ecx, 24             ← buffer size
    mov edx, 0x534D4150     ← magic 'SMAP'
    mov di, memory_map      ← where to store entries
    int 0x15
    jc  error               ← CF=1 means not supported

.loop:
    add di, 24              ← move to next buffer slot
    test ebx, ebx           ← is EBX zero?
```

```
    jz  .done              ← yes = last entry, done


    mov eax, 0x0000E820     ← function code again
    mov ecx, 24             ← buffer size
    mov edx, 0x534D4150     ← magic number
    int 0x15
    jc  .done              ← error = stop
    jmp .loop              ← get next entry


.done:
    ← all entries collected
```

Why This Must Be Done In Real Mode

The E820 function only works in Real Mode. The moment you switch to Protected Mode this function becomes completely unavailable.

This is why getting the memory map is one of the very first things Stage 2 does — before any CPU mode switching happens.

INT 0x15 Summary

EAX = 0xE820 → get memory map entry by entry

loop until EBX = 0

must call before leaving Real Mode

5.4 — INT 0x16: Keyboard Services

INT 0x16 handles keyboard input. Useful for bootloader menus or waiting for user input.

Function 0x00 — Read Keystroke

Waits for a key to be pressed and returns it.

Before calling:

AH = 0x00

After calling:

AH = scan code (hardware key code)

AL = ASCII value of key pressed

The call blocks — it waits until a key is actually pressed before returning.

Example — wait for any key:

```
mov ah, 0x00
int 0x16
← AH = scan code of key pressed
← AL = ASCII value of key pressed
```

Example — wait for key and print it:

```
mov ah, 0x00
int 0x16          ← wait for keypress
                  ← AL now has ASCII value
```

```
mov ah, 0x0E        ← print it

int 0x10
```

Function 0x01 — Check If Key Available

Checks if a key has been pressed without waiting.

Before calling:

AH = 0x01

After calling:

ZF = 1 → no key waiting in buffer

ZF = 0 → a key is available

AH = scan code if key available

AL = ASCII value if key available

Example — check if key pressed without blocking:

```
mov ah, 0x01

int 0x16

jz  no_key_pressed     ← ZF=1 means no key waiting ← if we get here, a key is waiting
```

INT 0x16 Summary

AH = 0x00 → wait for key press, return ASCII + scan code

AH = 0x01 → check if key available without waiting

The Golden Rules

Always put function code in AH before calling

Always check carry flag after INT 0x13 disk operations

Always get memory map (INT 0x15 E820) before leaving Real Mode

INT 0x10 and INT 0x13 are your most used tools

Once you switch CPU modes all of these are gone forever